

ООП - програмна парадигма, при която системата се моделира
като набор от обекти, които взаимодействат помежду си.
→ фундаментален стил на програмиране

Принципи на ООП

интерфейс в програмирането - всичко, което един обект дава на
(неформално) външния свят и може да използва

• Абстракция - скриване на имплементацията и ненужните детайли
от потребителя; той работи с интерфейса на обекта
без да знае как е имплементиран

• Капсулация - "скриване на информация"; потребителите на обект
не могат да променят неговото вътрешно състояние
по неохотен начин

• Наследяване - могат да бъдат дефинирани и създавани обекти, които
са специализирани варианти на вече съществуващи
обекти; Клас наследник взема всичко на класа-родител.

• Полиморфизъм - обекти от един и същи тип имат един и същи
интерфейс, но с различна реализация на този интерфейс

1. Структури - "пакет" от свързани данни

Как да имаме структура, която съдържа член от същия тип:

```
① struct Node {  
    int value;  
    Node* next = nullptr; ← Указател към следващия елемент  
};
```

⊕ Знаем размера на Node, защото next е указател (адрес)
Позволява динамични списъци с данни - дървета и свързани списъци

⊖ При динамично заделяне → ръчно управление на паметта

2) struct Node {
 int value;
 Node& next;

!!! Задължително: инициализира се
 Референция към друг Node

Node(int val, Node& n): value(val), next(n) {}

};
 няма default конструктор!!!

Node last = {20, last} // dummy референция
 тоест към себе си

⊖ неудобен при динамични структури
 не може да се променя, защото референциите са неизменяеми
 няма default конструктор

⊕ използва се когато винаги ще има Node, към който да се сочи

Представяне в паметта и подравняване

- В паметта променливите са разположени по същия начин, в който са дефинирани в структурата
- Подравняването става по големината на най-голямата елемент-данна
- Минимална памет = когато елемент-данните в структурата са подредени в намаляващ / нарастващ ред

* sizeof(празна = няма елемент-данни структура) = 1

Little-endian - най-старшият байт на последна позиция,
 най-младшият - на първа

Big-endian - най-старшият е на първа позиция,
 най-младшият на последна

Обединение (unions) - вид структура, чиито елементи споделят една и съща памет

```
union Example {  
    int a;  
    char b;  
};
```

```
Example var.a = 65;  
cout << var.a << " " << var.b;  
65 A  
sizeof(union Example) = 4
```

Енумерации (enums)

Enum class
↓
scoped
стойностите не се преобразуват имплицитно към други типове

Enum
↓
unscoped
стойностите се преобразуват —
...

Namespaces - метод за предотвратяване на конфликти с имена

Class - подобно на struct

- при големи данни
- private

- "обект с комплексно поведение и по-ясен интерфейс"

Обект - конкретна променлива/инстанция от тип (struct/class)